Pulse

Real-Time Structural Feedback for AI Code Generation

Omar Saidahmed

April 2026

Abstract. Pulse is a code smell detector that runs as a post-edit hook for AI coding agents. After every file modification, it checks the changed file for 26 structural code smells and blocks the agent from continuing until the findings are resolved. Recent research shows that AI-assisted development leads to persistent increases in code complexity (+41%) and static analysis warnings (+30%), with the accumulated technical debt cancelling out initial velocity gains within two months. Pulse intervenes at the point of introduction, the individual edit, rather than after the damage compounds. This paper describes the problem, why existing tools fail to address it for agents, the architecture and threshold methodology behind Pulse, and what remains unknown.

1 The problem

AI coding agents write code that works. They also write code that rots.

He et al. (2026) studied 806 GitHub repositories that adopted Cursor, comparing them against 1,380 matched controls using a difference-in-differences design [1]. The results were stark: a 281% spike in lines added during the first month, collapsing to baseline by month three. Static analysis warnings rose 30% and code complexity rose 41%, and both stayed elevated permanently. Their panel GMM models showed the accumulated technical debt then slowed future development velocity, feeding back into itself. The velocity gains from AI adoption were fully cancelled by a 3x increase in code complexity.

Xu et al. (2026) reach a similar conclusion through a different methodology. Using a difference-in-differences design on Microsoft-owned OSS projects after Copilot's launch, they found that peripheral contributors increased commits by 43.5% and pull requests by 17.7%, while core contributors reduced their original code productivity by 19% and increased their review burden by 6.5% [2]. PR rework rose 2.4%. The productivity gains land on less-experienced developers; the maintenance cost lands on the experienced ones. He's panel-data signal and Xu's social-cost signal point at the same underlying problem from opposite ends.

This is not specific to Cursor or Copilot. Chen et al. (2026) tested 18 AI models across 100 Python repositories, each spanning an average of 233 days and 71 consecutive commits. Most models hit a zero-regression rate below 0.25, which means they broke previously passing tests in 75% or more of maintenance iterations [3]. SWE-CI, the benchmark they used, asks a simple question: can an agent maintain a codebase over time without breaking things? For most models, the answer is no.

There is a structural explanation. When an agent writes a 200-line function with cyclomatic complexity of 25 and nesting depth of 6, it creates something that is hard to change safely. The next invocation that touches that function has to navigate 25 independent paths through the control flow. Get one wrong, and tests break. And each modification that adds complexity makes the next modification riskier.

Borg et al. (2026) showed this connection directly. They analyzed 5,000 Python files and found language models fail 15–30% more often when modifying unhealthy code compared to healthy code [4]. Their conclusion was blunt: "human-friendly code is also more compatible with AI tooling." Code health at or above 9.5 on CodeScene's 10-point scale is where agents operate reliably. Below that, defect risk rises, and the relationship is non-linear [5].

Agents do refactor. They just don't refactor structure. Horikawa et al. (2025) studied 15,451 agentic refactorings across 12,256 pull requests in real Java projects and found that 35.8% of agent refactorings are low-level edits like renames and type changes, compared to 24.4% for humans, while only 43.0% are high-level structural changes versus 54.9% for humans [6]. The median delta in design and implementation smell counts before and after agentic refactoring is zero. So the fragility introduced at write time does not get cleaned up later by the agent itself, which closes the loop on why complexity accumulates rather than dissipates.

The chain looks like this:

Agent writes fragile code → Later changes touch fragile code → Tests break

SWE-CI measures the end of that chain. He et al. measured the accumulation in the middle. Pulse operates at the beginning.

1.1 What existing tools miss

Static analysis tools exist for every major language. Clippy for Rust, ESLint for JavaScript, Pylint for Python. They run in CI or as editor plugins. For AI agents, this falls apart for three reasons.

The first is timing. A CI pipeline that dumps 40 findings after a 500-line commit hands the agent a list of problems in a structure it already committed to. Restructuring at that point means undoing architectural decisions, not patching local issues. He et al.'s data shows this: complexity accumulates month over month because nothing catches it at the point of introduction.

The second is enforcement. Sadowski et al. (2018) documented this at Google: their Tricorder platform hit 95% actionability only by disabling any analyzer where "Not useful" clicks exceeded 10% [7]. For agents, the dynamic is worse. A warning enters the context window, competes for attention against the task the agent is trying to finish, and loses. Agents act on blocking errors. They skip warnings.

The third is compliance. CodeScene and SonarQube now offer MCP servers that expose code health analysis as tools agents can call. The approach is flexible but voluntary. The agent has to decide to invoke the tool. CodeScene's own documentation acknowledges this: "When left on their own, AI agents often invoke tools inconsistently or skip important safeguards." The workaround is writing instructions (in AGENTS.md or CLAUDE.md files) telling the agent when to call the tool. This works when the agent follows instructions. It fails when the agent is under token pressure or deep in a complex task.

And even when agents do call review tools, the result is not necessarily cleaner code. Zhong et al. (2026) analyzed 278,790 review conversations across 300 GitHub projects and found that suggestions made by AI agent reviewers are adopted into the codebase at 16.6% versus 56.5% for human reviewers, and that adopted AI suggestions produce significantly larger increases in code complexity and code size than human ones [8]. Over half of the unadopted AI suggestions were either incorrect or addressed through alternative fixes. So the failure mode for review-style MCP tools has two halves: agents skip them, and when they don't skip them, the suggestions they follow tend to add structure rather than reduce it.

2 Pulse

Pulse is a compiled Rust binary that parses source files with tree-sitter grammars. It covers 22 languages: Python, TypeScript, JavaScript, Rust, C, C++, Java, C#, Go, Swift, Zig, Ruby, Objective-C, Tcl, Kotlin, Haskell, Lua, R, PHP, COBOL, D, and Groovy.

Three modes of operation:

- `pulse check <file>` – verbose analysis for manual review.
- `pulse --hook` – the PostToolUse hook. Reads JSON from stdin, analyzes the file, emits blocking findings. This is how agents interact with it.
- `pulse --stop` – session-end check. Compares final state against baselines, flags regressions.

Analysis runs in under 10 ms per file. Clean files and unsupported types produce no output.

2.1 Why a hook, not an MCP tool

A PostToolUse hook and an MCP tool solve the same problem with different enforcement models. The relevant comparison is reliability, not flexibility.

An MCP tool waits to be called. The agent writes code, then maybe checks quality, then maybe acts on the result. Each "maybe" is a point of failure. A PostToolUse hook fires automatically after every file edit. The agent does not choose to run it, cannot skip it, and cannot proceed until findings are resolved.

This matters because the problem He et al. identified is one of accumulation. Complexity does not spike in a single edit. It creeps up over dozens. An MCP tool that the agent forgets to call on edit 14 of 30 lets that edit's complexity through. A hook catches it regardless.

The trade-off is that hooks are less flexible. An MCP tool can provide context, answer questions, and suggest refactoring strategies. A hook can only say "this is wrong, fix it." For the specific problem of preventing structural degradation, the blunt enforcement model is more reliable. But it is worth being honest that this is a trade-off, not a pure win.

The design also matches what practitioners ask for. Liao et al. (2026) surveyed industrial developers on adoption preferences for static defect detection and found that 100% preferred integration into the IDE rather than CI, 31.8% specifically wanted detection to fire after every code modification, and 54.5% preferred change-level granularity over file-level [9]. Their context was performance anti-patterns rather than structural smells, but the preference shape is the same. A blocking PostToolUse hook is the closest fit to that preference.

2.2 Three-tier architecture

Detections fire at different points during a session.

The first tier fires after every edit. The PostToolUse hook reports function-level findings near the edited range. If an agent modifies lines 50–80, it only sees findings for functions overlapping that range. On the first write of a new file, module-level findings are included too.

The second tier runs as a periodic checkpoint. Every 2nd edit on new files and every 5th on existing files, a module-level regression check runs. If the file got worse since the session started, those regressions are reported.

The third tier runs once at session end. The stop hook does a final regression sweep across every file touched during the session.

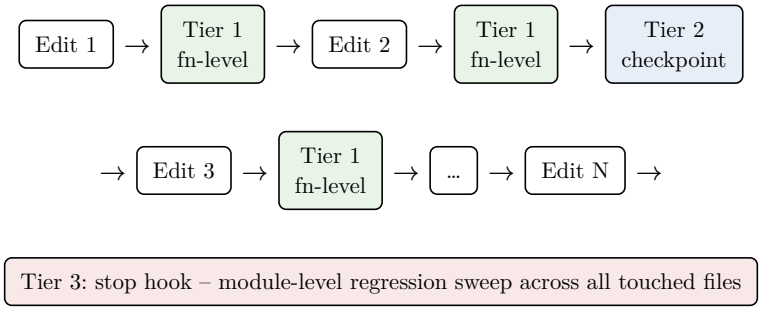

Figure 1: Three-tier detection timeline. Green nodes fire after every edit. Blue nodes fire periodically. The red node fires once at session end.

Every hook output includes a budget line showing remaining headroom:

```
[budget] fn=18/20 loc=340/500 cc=75/100
```

When function count and per-function complexity are both near their limits, a conflict note appears:

```
[conflict] fn count and per-function complexity are both constrained
         – merge only low-cc functions
```

That conflict note exists because of a failure mode I observed during development. Agents get stuck oscillating: Pulse says "too many functions," the agent merges two, Pulse says "complex method" on the now-larger function, the agent splits it back out. The conflict note tells the agent which direction is safe before it starts guessing.

2.3 Session isolation

Each session gets its own baseline directory. Without this, concurrent sessions would corrupt each other's regression tracking.

2.4 Configuration

Projects can customize thresholds via a `.pulse.toml` file in the project root. All fields are optional, and missing values use defaults.

```toml
[thresholds]
arg_max = 8
fn_loc_warning = 80

[disable]
smells = ["primitive_obsession"]

[languages.go]
arg_max = 7
```

The config supports per-language threshold overrides and smell disabling. Pulse searches up the directory tree from the analyzed file, the same discovery pattern as `Cargo.toml` or `.eslintrc`.

3 What Pulse detects

Pulse detects 26 structural code smells in two categories. Function-level smells (14) fire on individual functions: complexity, size, nesting depth, argument count, duplication, empty error handlers, and similar structural issues. Module-level smells (12) fire on whole files: total size, function count, aggregate complexity, cohesion, code duplication, and struct field counts.

Every smell maps to a metric computable from the AST in a single pass. No type resolution, no cross-file analysis, no external dependencies. The full catalog is in the project repository.

The two tiers exist for a reason. Cotroneo et al. (2025) compared over 500,000 AI-generated and human-written code samples across Python and Java and found that AI-generated functions are typically simpler in isolation: lower cyclomatic complexity, fewer tokens, more repetitive structure [10]. On the surface this contradicts He et al.'s longitudinal complexity rise. It does not. Snippet-level simplicity coexists with repository-level complexity rise because the accumulating complexity is not in any single function. It lives in the count, the duplication, and the connections between functions. Function-level smells alone would miss it. Module-level smells alone would catch the aggregate but miss the local pathologies. Pulse runs both.

4 Threshold methodology

Every threshold in Pulse comes from peer-reviewed research. A review of 35 papers was done to validate or challenge the values.

Metric	**Threshold**	**Source**	**Evidence**
Cyclomatic complexity	9 warn, 18 alert	McCabe (1976), NIST SP 500-235	Moderate. CC=10 has historical backing but is mostly a proxy for LOC
Cognitive complexity	15 warn, 25 alert	Campbell (2016), ESEM 2020	Moderate. Validated against comprehension time in 24,000 evaluations
Function LOC	65 warn, 100 alert	Withrow (1990), Hatton (1997)	Moderate. Academic sweet spot is 150–250 LOC; 65 bal-

Metric	Threshold	Source	Evidence
			ances that with practice
File LOC	500 warn, 700 alert	Yamashita et al. (2016), Hatton (1997)	Weak. Splitting into smaller files "may be counterproductive"
Parameter count	5	Practitioner consensus	Weak. No empirical validation found
Nesting depth	4	Cognitive load research (2023)	Moderate. Supported by hierarchical complexity theory
Code duplication	6 LOC exact, 20 LOC fuzzy	Juergens et al. (ICSE 2009)	Strong. 52% of clones inconsistently changed; 15% caused faults
LCOM4 cohesion	3 components	Hitz & Montazeri (1996)	Weak–Moderate. Best-supported variant but confounded by class size

Two things from the literature review are worth calling out. Jay et al. (2009) analyzed 1.2 million files and found LOC predicts about 90% of CC's variance [14]. CC is a useful shorthand, but it rarely catches something that function length does not. And Palomba et al. (2018) found smelly classes are more change-prone and fault-prone, but smells are "not necessarily a direct cause of faults, but a co-occurring phenomenon" [18]. I want to be clear about what this means for Pulse: it does not claim to prevent bugs. It prevents structural conditions that make bugs more likely when code gets modified later.

5 Observations from development

Pulse was developed iteratively using itself. Over 10 sessions, independent Claude Code instances each implemented a tree-sitter language walker under Pulse's enforcement. After each implementation, the session analyzed its own interaction with the tool.

Three findings shaped the tool's design.

The first was constraint conflict oscillation. 8 of 10 sessions hit the same loop: too many functions, so the agent merges some. Now a function is too complex, so the agent splits it back out. One session went through 6 rewrites on a single file. The root cause is that reducing function count and reducing per-function complexity are inversely correlated when a file is near both limits. The fix was a conflict note that fires when both constraints are tight, telling the agent which merges are safe (functions with cc of 3 or less).

The second was late module feedback. The original design deferred module-level findings to the stop hook at session end. An agent could write a 500-line, 25-function file and get no signal until the session ended. By then, restructuring was expensive. The fix was to report module-level findings on the first write of any new file.

The third was budget invisibility. Half the sessions skipped the `pulse budget --new` pre-check despite instructions requiring it. The agent's task-completion drive dominated over the compliance instruction. The fix had two parts: a budget line now appears in every hook output so the agent sees headroom whether it asked or not, and the instruction was rewritten from advisory to procedural.

These observations come with a caveat. All 10 sessions implemented tree-sitter walkers, which is AST-heavy, dispatch-heavy code that is structurally repetitive across languages. A web application or a data pipeline would exercise different smell distributions. The findings are real but skewed toward the patterns this particular codebase produces.

A proper A/B evaluation (same task, same model, with and without Pulse, measuring regression rates over time) has not been done. The effectiveness claims here are mechanistic arguments backed by correlational evidence, not experimental proof. He et al.'s methodology, difference-in-differences with matched controls, applied to Pulse-adopting versus non-adopting repositories would be the right study design.

6 Limitations

Pulse prevents structural fragility. It does not prevent bugs.

Semantic errors are out of scope. A 10-line function with CC of 2 can still compute the wrong answer. Pulse knows nothing about what code means.

Context loss is out of scope. Agents forget design decisions across sessions. Clean structure helps (it is easier to re-read) but does not replace memory.

API misuse is out of scope. Calling a library function with the wrong arguments will not trigger a smell if the argument count looks normal.

Concurrency bugs are out of scope. Race conditions, deadlocks, and ordering problems are invisible to structural analysis.

Type system problems are out of scope. Using a raw string where an enum belongs, missing a trait implementation, or annotating a wrong lifetime would all need type resolution that tree-sitter cannot provide.

The feedback loop also has a cost. Pulse checks code after the agent writes it, not while it is being written. The agent generates, Pulse evaluates, findings come back, the agent rewrites. This loop converges, but not always fast. The generate-evaluate-rewrite architecture is not specific to structural smells: Zhu et al. (2025) use static AST pattern matching to steer code generation toward efficiency [23], and Mathews and Nagappan (2024) show that iterative test-feedback loops improve LLM correctness on standard benchmarks [24]. The signal differs across these systems; the loop shape does not. Injecting constraints into the generation step directly would be more efficient, but that requires changes at the agent framework level, not the tool level.

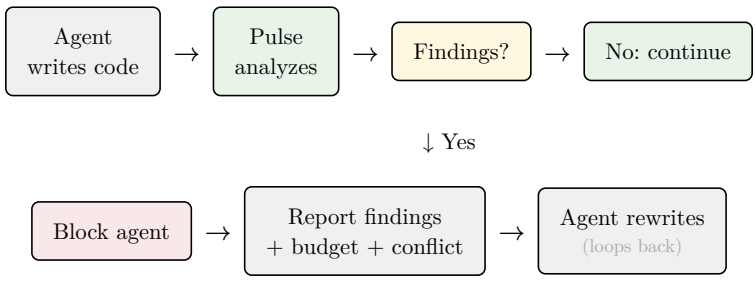

Figure 2: The generate-then-check loop. Pulse evaluates after the write, not during. Budget and conflict lines are included in the output to help the agent converge faster.

7 Conclusion

He et al. showed that AI-assisted development produces a persistent 41% increase in code complexity and that this accumulated debt cancels out the initial velocity gains. Borg et al. showed that unhealthy code causes AI agents to fail 15–30% more often. Chen et al. showed that 75% of AI agents break previously working code during long-term maintenance.

Pulse addresses the structural layer of this problem. Not the semantic layer, not the context layer, not the type-safety layer. The structural layer: preventing agents from building the fragile code that later breaks. It is the most tractable layer to enforce automatically because it does not require understanding business logic, accessing external systems, or maintaining cross-session memory. It only requires that the code, at every step, stays small enough to understand, simple enough to modify, and modular enough to change without breaking something else.

Whether this intervention actually prevents the complexity accumulation He et al. measured is an open empirical question. The tool exists. The methodology for testing it exists. The study has not been done.

Pulse is open source at github.com/osaidahmed/pulse.

References

[1] H. He, C. Miller, S. Agarwal, C. Kästner, and B. Vasilescu. "Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects." *Proc. MSR '26*, April 2026.

[2] F. Xu, P. K. Medappa, M. M. Tunc, M. Vroegindeweij, and J. C. Fransoo. "AI-Assisted Programming Decreases the Productivity of Experienced Developers by Increasing the Technical Debt and Maintenance Burden." *arXiv:2510.10165*, January 2026.

[3] J. Chen, X. Xu, H. Wei, C. Chen, and B. Zhao. "SWE-CI: Evaluating Agent Capabilities in Maintaining Codebases via Continuous Integration." *arXiv:2603.03823*, March 2026.

[4] M. Borg, N. Hagatulah, A. Tornhill, and E. Söderberg. "Code for Machines, Not Just Humans: Quantifying AI-Friendliness with Code Health Metrics." *arXiv:2601.02200*, accepted at FORGE 2026.

[5] A. Tornhill. "AI-Ready Code: How Code Health Determines AI Performance." CodeScene Whitepaper, January 2026.

[6] K. Horikawa, H. Li, Y. Kashiwa, B. Adams, H. Iida, and A. E. Hassan. "Agentic Refactoring: An Empirical Study of AI Coding Agents." November 2025.

[7] C. Sadowski, E. Aftandilian, A. Eagle, L. Miller-Cushon, and C. Jaspan. "Lessons from Building Static Analysis Tools at Google." *Communications of the ACM* 61(4), 2018.

[8] S. Zhong, S. Noei, Y. Zou, and B. Adams. "Human-AI Synergy in Agentic Code Review." *arXiv:2603.15911*, March 2026.

[9] C. Sporea, A. Toma, and S. Sajedi. "On the Practical Adoption of a Static Performance Anti-Pattern Detector: An Industrial Case Study." 2026.

[10] D. Cotroneo, C. Improta, and P. Liguori. "Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity." *arXiv:2508.21634*, August 2025.

[11] T. J. McCabe. "A Complexity Measure." *IEEE Transactions on Software Engineering* SE-2(4), 1976.

[12] A. H. Watson and T. J. McCabe. "Structured Testing: A Testing Methodology Using the Cyclomatic Complexity Metric." *NIST SP 500-235*, 1996.

[13] G. A. Campbell. "Cognitive Complexity: An Overview and Evaluation." *ACM International Conference on Technical Debt*, 2018.

[14] G. Jay, J. Hale, R. Smith, D. Hale, N. Kraft, and C. Ward. "Cyclomatic Complexity and Lines of Code: Empirical Evidence of a Stable Linear Relationship." *Journal of Software Engineering and Applications* 2(3), 2009.

[15] M. Munoz Baron, M. Wyrich, and S. Wagner. "An Empirical Validation of Cognitive Complexity as a Measure of Source Code Understandability." *14th ACM/IEEE ESEM* (Best Full Paper), 2020.

[16] C. Withrow. "Error Density and Size in Ada Software." *IEEE Software*, 1990.

[17] L. Hatton. "Reexamining the Fault Density–Component Size Connection." *IEEE Software* 14(2), 1997.

[18] F. Palomba, G. Bavota, M. Di Penta, F. Ferrucci, A. De Lucia, and R. Oliveto. "On the Diffuseness and the Impact on Maintainability of Code Smells." *Empirical Software Engineering* 23, 2018.

[19] E. Juergens, F. Deissenboeck, and B. Hummel. "Do Code Clones Matter?" *31st International Conference on Software Engineering*, 2009.

[20] D. I. K. Sjoberg, A. Yamashita, B. C. D. Anda, A. Mockus, and T. Dyba. "Quantifying the Effect of Code Smells on Maintenance Effort." *IEEE Transactions on Software Engineering*, 2013.

[21] K. Yamashita, C. Huang, M. Nagappan, Y. Kamei, A. Mockus, A. E. Hassan, and N. Ahmed. "Thresholds for Size and Complexity Metrics." *IEEE QRS*, 2016.

[22] M. Hitz and B. Montazeri. "Measuring Coupling and Cohesion in Object-Oriented Systems." *Proc. Int'l Symposium on Applied Corporate Computing*, 1996.

[23] D. Zhu, D. Chen, J. Chen, J. Grossklags, A. Pretschner, and W. Shang. "More Than Just Functional: LLM-as-a-Critique for Efficient Code Generation." 2025.

[24] N. S. Mathews and M. Nagappan. "Test-Driven Development and LLM-based Code Generation." *ASE '24*, 2024.